

```
<autoResourcePackage qualifier="DXFeatureLevel"/>
</packaging>
```

This example shows how to use manual `resourcePackage`.

XML

```
<packaging>
  <resourcePackage name="Germany">
    <qualifierSet definition="lang-de-de"/>
    <qualifierSet definition="lang-es-es"/>
  </resourcePackage>
  <resourcePackage name="France">
    <qualifierSet definition="lang-fr-fr"/>
  </resourcePackage>
  <resourcePackage name="HighRes1">
    <qualifierSet definition="scale-200"/>
  </resourcePackage>
  <resourcePackage name="HighRes2">
    <qualifierSet definition="scale-400"/>
  </resourcePackage>
</packaging>
```

MakePri.exe doesn't explicitly block generation of resource PRI files along any specific dimension. Restrictions along a certain set of dimensions are defined and implemented externally by either MakeAppx.exe or other tools in the pipeline.

MakePri.exe parses the `packaging` element after all the `index` nodes to populate all the default qualifiers. MakePri.exe collects parsed info in these data structures.

C#

```
enum ResourcePackageMode
{
    None,
    AutoPackQualifier,
    ManualPack
}

ResourcePackageMode eResourcePackageMode;
list<string> RPQualifierList; // To store AutoResourcePackage Qualifiers
map<string, list<string>> RPNameToQSIMap; // To store ResourcePackage name
to QualifierSet list mapping.
```

resources@isDeploymentMergeable attribute

This attribute sets a flag in the PRI file that causes

- Deployment merge to identify that this PRI file can merge.
- GetFullyQualifiedReference to return an error in case this flag is set and the resource manager has been initialized with a file.

The default value of this attribute is `true`. MakePri.exe only sets the flag in PRI if you target Windows 10.

We recommend that you omit `isDeploymentMergeable` (or set it explicitly to `true`) for resource pack creation if you target Windows 10.

MakePri.exe adds the value of `isDeploymentMergeable` to the dump file if `makepri dump` is run with the `/dt detailed` option.

XML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<PriInfo>
  <PriHeader>
    ...
    <IsDeploymentMergeable>true</IsDeploymentMergeable>
    ...
  </PriHeader>
  ...
</PriInfo>
```

resources@majorVersion attribute

The default value for this attribute is 1. If you provide an explicit value, and you also use the deprecated `/VersionMajor(vma)` command-line option for the MakePri.exe tool, then the value in the config file takes precedence.

Here's an example.

XML

```
<resources majorVersion="2">
  <packaging ... />
  <index root="" startIndexAt="">
    ...
  </index>
</resources>
```

resources@targetOsVersion attribute

Indicates the target operating system version. The table below shows the values that are supported; the default value is 6.3.0.

Value	Meaning
10.0.0	Windows 10
6.3.0 (default)	Windows 8.1
6.2.1	Windows 8

Here's an example.

XML

```
<resources targetOsVersion="10.0.0">
  <packaging ... />
  <index root="" startIndexAt="">
    ...
  </index>
</resources>
```

Note Windows is backward compatible with respect to PRI files; but not always forward compatible.

MakePri.exe adds the value of `targetOsVersion` to the dump file if `makepri dump` is run with the `/dt detailed` option.

XML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<PriInfo>
  <PriHeader>
    ...
    <TargetOS version="10.0.0"/>
    ...
  </PriHeader>
  ...
</PriInfo>
```

Validation error messages

Here are some example error conditions, and the corresponding error message.

Condition	Severity	Message
-----------	----------	---------

Condition	Severity	Message
A targetOsVersion other than one of the supported values is specified.	Error	Invalid Configuration: Invalid targetOsVersion specified.
A targetOsVersion of "6.2.1" is specified and a <code>packaging</code> element is present.	Error	Invalid Configuration: 'Packaging' node is not supported with this targetOsVersion.
More than one mode found in the configuration. For example, Manual and AutoResourcePackage specified.	Error	Invalid Configuration: 'packaging' node cannot have more than one mode of operation.
A default qualifier is listed under resource package.	Error	Invalid Configuration: <Qualifiername> = <QualifierValue> is a default qualifier and its candidates cannot be added to a resource package.
AutoResourcePackage qualifier contains multiple qualifiers. For example, language_scale.	Error	Invalid Configuration : AutoResourcePackage with multiple qualifiers is not supported.
ResourcePackage QualifierSet contains multiple qualifiers. For example, language-en-us_scale-100	Error	Invalid Configuration : QualifierSet with multiple qualifiers is not supported.
Duplicate resourcepack name found.	Error	Invalid Configuration : Duplicate resource pack name <rpname>.
Same qualifier set defined in two resource packages.	Error	Invalid Configuration: Multiple instances of QualifierSet "<qualifier tags>" found.
No candidates are found for the QualifierSet listed for 'ResourcePackage' node.	Warning	Invalid Configuration: No candidates found for <Resource Package Name>.
No candidates found for qualifier listed under 'AutoResourcePackage' node.	Warning	Invalid Configuration: No candidates found for qualifier <qualifier name>. Resource Package not generated.
None of the modes are found. That is, empty 'packaging' node found.	Warning	Invalid Configuration: No packaging mode specified.

Related topics

- [Compile resources manually with MakePri.exe](#)
- [MakePri.exe command-line options—createconfig command](#)
- [Tailor your resources for language, scale, high contrast, and other qualifiers](#)
- [Resource Management System—ResourceContext](#)

MakePri.exe format-specific indexers

Article • 10/20/2022

This topic describes the format-specific indexers used by the [MakePri.exe](#) tool to generate its index of resources.

ⓘ Note

MakePri.exe is installed when you check the **Windows SDK for UWP Managed Apps** option while installing the Windows Software Development Kit. It is installed to the path `%WindowsSdkDir%bin\<WindowsTargetPlatformVersion>\x64\makepri.exe` (as well as in folders named for the other architectures). For example, `C:\Program Files (x86)\Windows Kits\10\bin\10.0.17713.0\x64\makepri.exe`.

MakePri.exe is typically used with the `new`, `versioned`, or `resourcepack` commands. See [MakePri.exe command-line options](#). In those cases it indexes source files to generate an index of resources. MakePri.exe uses various individual indexers to read different source resource files or containers for resources. The simplest indexer is the folder indexer, which indexes the contents of a folder, such as `.jpg` or `.png` images.

You identify format-specific indexers by specifying `<indexer-config>` elements within an `<index>` element of the [MakePri.exe configuration file](#). The `type` attribute identifies the format-specific indexer that is used.

Resource containers encountered during indexing usually get their contents indexed rather than being added to the index themselves. For example, `.resjson` files that the folder indexer finds may be further indexed by a `.resjson` indexer, in which case the `.resjson` file itself does not appear in the index. **Note** an `<indexer-config>` element for the indexer associated with that container must be included in the configuration file for that to happen.

Typically, qualifiers found on a containing entity—such as a folder or a `.resw` file—are applied to all resources within it, such as the files within the folder, or the strings within the `.resw` file.

Folder

The folder indexer is identified by a `type` attribute of `FOLDER`. It indexes the contents of a folder, and determines resource qualifiers from the folder and filenames. Its

configuration element conforms to the following schema.

XML

```
<xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:simpleType name="ExclusionTypeList">
    <xs:restriction base="xs:string">
      <xs:enumeration value="path"/>
      <xs:enumeration value="extension"/>
      <xs:enumeration value="name"/>
      <xs:enumeration value="tree"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="FolderExclusionType">
    <xs:attribute name="type" type="ExclusionTypeList" use="required"/>
    <xs:attribute name="value" type="xs:string" use="required"/>
    <xs:attribute name="doNotTraverse" type="xs:boolean"
use="required"/>
    <xs:attribute name="doNotIndex" type="xs:boolean" use="required"/>
  </xs:complexType>
  <xs:simpleType name="IndexerConfigFolderType">
    <xs:restriction base="xs:string">
      <xs:pattern value="((f|F)(o|O)(l|L)(d|D)(e|E)(r|R))"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:element name="indexer-config">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="exclude" type="FolderExclusionType"
minOccurs="0" maxOccurs="unbounded"/>
      </xs:sequence>
      <xs:attribute name="type" type="IndexerConfigFolderType"
use="required"/>
      <xs:attribute name="foldernameAsQualifier" type="xs:boolean"
use="required"/>
      <xs:attribute name="filenameAsQualifier" type="xs:boolean"
use="required"/>
      <xs:attribute name="qualifierDelimiter" type="xs:string"
use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

The `qualifierDelimiter` attribute specifies the character after which qualifiers are specified in a filename, ignoring the extension. The default is ".".

PRI

The PRI indexer is identified by a `type` attribute of PRI. It indexes the contents of a PRI file. You typically use it when indexing the resource contained within another assembly, DLL, SDK, or class library into the app's PRI.

All resource names, qualifiers and values contained in the PRI file are directly maintained in the new PRI file. The top level resource map, however is not maintained in the final PRI. Resource Maps are merged.

XML

```
<xs:schema id="prifile" xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">
  <xs:simpleType name="IndexerConfigPriType">
    <xs:restriction base="xs:string">
      <xs:pattern value="((p|P)(r|R)(i|I))"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:element name="indexer-config">
    <xs:complexType>
      <xs:attribute name="type" type="IndexerConfigPriType"
use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

PriInfo

The PriInfo indexer is identified by a `type` attribute of PRIINFO. It indexes the contents of a detailed dump file. You produce a detailed dump file by running `makepri dump` with the `/dt detailed` option. The configuration element for the indexer conforms to the following schema.

XML

```
<xs:schema id="priinfo" xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">
  <xs:simpleType name="IndexerConfigPriInfoType">
    <xs:restriction base="xs:string">
      <xs:pattern value="((p|P)(r|R)(i|I)(i|I)(n|N)(f|F)(o|O))"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:element name="indexer-config">
    <xs:complexType>
      <xs:attribute name="type" type="IndexerConfigPriInfoType"
use="required"/>
      <xs:attribute name="emitStrings" type="xs:boolean" use="optional"/>
      <xs:attribute name="emitPaths" type="xs:boolean" use="optional"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```
</xs:element>
</xs:schema>
```

This configuration element allows for optional attributes to configure the behavior of the PriInfo indexer. The default value of `emitStrings` and `emitPaths` is `true`. If `emitStrings` is `true` then resource candidates with the `type` attribute set to "String" are included in the index, otherwise they are excluded. If 'emitPaths' is `true` then resource candidates with the `type` attribute set to "Path" are included in the index, otherwise they are excluded.

Here is an example configuration that includes String resource types but skips Path resource types.

XML

```
<indexer-config type="priinfo" emitStrings="true" emitPaths="false" />
```

To be indexed, a dump file must end with the extension `.pri.xml`, and must conform to the following schema.

XML

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  elementFormDefault="qualified" >
  <xs:simpleType name="candidateType">
    <xs:restriction base="xs:string">
      <xs:pattern value="Path|String"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="scopeType">
    <xs:sequence>
      <xs:element name="ResourceMapSubtree" type="scopeType" minOccurs="0"
        maxOccurs="unbounded"/>
      <xs:element name="NamedResource" minOccurs="0" maxOccurs="unbounded">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="Decision" minOccurs="0" maxOccurs="unbounded">
              <xs:complexType>
                <xs:sequence>
                  <xs:any processContents="skip" minOccurs="0"
                    maxOccurs="unbounded"/>
                </xs:sequence>
              </xs:complexType>
            </xs:element>
            <xs:element name="Candidate" minOccurs="0"
              maxOccurs="unbounded">
                <xs:complexType>
```



```

        <xs:sequence>
            <xs:element name="QualifierSet" minOccurs="0"
maxOccurs="unbounded">
                <xs:complexType>
                    <xs:sequence>
                        <xs:element name="Qualifier" minOccurs="0"
maxOccurs="unbounded">
                            <xs:complexType>
                                <xs:attribute name="name" type="xs:string"
use="required" />
                                <xs:attribute name="value" type="xs:string"
use="required" />
                                <xs:attribute name="priority" type="xs:integer"
use="required" />
                                <xs:attribute name="scoreAsDefault"
type="xs:decimal" use="required" />
                                <xs:attribute name="index" type="xs:integer"
use="required" />
                            </xs:complexType>
                        </xs:element>
                    </xs:sequence>
                    <xs:anyAttribute processContents="skip" />
                </xs:complexType>
            </xs:element>
            <xs:element name="Value" type="xs:string" minOccurs="0"
maxOccurs="unbounded"/>
        </xs:sequence>
        <xs:attribute name="type" type="candidateType"
use="required" />
    </xs:complexType>
</xs:element>
</xs:sequence>
<xs:attribute name="name" use="required" type="xs:string" />
<xs:anyAttribute processContents="skip" />
</xs:complexType>
</xs:element>
</xs:sequence>
<xs:attribute name="name" use="required" type="xs:string" />
<xs:anyAttribute processContents="skip" />
</xs:complexType>
<xs:element name="PriInfo">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="PriHeader" >
                <xs:complexType>
                    <xs:sequence>
                        <xs:any minOccurs="0" maxOccurs="unbounded"
processContents="skip" />
                    </xs:sequence>
                    <xs:anyAttribute processContents="skip" />
                </xs:complexType>
            </xs:element>
            <xs:element name="QualifierInfo">
                <xs:complexType>
                    <xs:sequence>

```

```

        <xs:any minOccurs="0" maxOccurs="unbounded"
processContents="skip" />
    </xs:sequence>
</xs:complexType>
</xs:element>
<xs:element name="ResourceMap">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="VersionInfo">
                <xs:complexType>
                    <xs:anyAttribute processContents="skip" />
                </xs:complexType>
            </xs:element>
            <xs:element minOccurs="0" maxOccurs="unbounded"
name="ResourceMapSubtree" type="scopeType" />
        </xs:sequence>
        <xs:attribute name="name" type="xs:string" use="required" />
        <xs:anyAttribute processContents="skip" />
    </xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>

```

MakePri.exe supports dump types 'Basic', 'Detailed', 'Schema', and 'Summary'. To configure MakePri.exe to emit the dump type that the PriInfo indexer can read, include "/DumpType Detailed" when using the `dump` command.

Several elements of the `.pri.xml` file are skipped by MakePri.exe. These elements are either computed during indexing, or are specified in the MakePri.exe configuration file. Resource names, qualifiers, and values that are contained within the dump file are directly maintained in the new PRI file. The top-level resource map, however, is not maintained in the final PRI. Resource Maps are merged as part of the indexing.

This is an example of a valid String type resource from a dump file.

XML

```

<NamedResource name="SampleString " index="96" uri="ms-
resource://SampleApp/resources/SampleString ">
    <Decision index="2">
        <QualifierSet index="1">
            <Qualifier name="Language" value="EN-US" priority="900"
scoreAsDefault="1.0" index="1"/>
        </QualifierSet>
    </Decision>
    <Candidate type="String">
        <QualifierSet index="1">
            <Qualifier name="Language" value="EN-US" priority="900"
scoreAsDefault="1.0" index="1"/>
        </QualifierSet>
    </Candidate>
</NamedResource>

```

```

    </QualifierSet>
    <Value>A Sample String Value</Value>
  </Candidate>
</NamedResource>

```

This is an example of a valid Path type resource with two candidates from a dump file.

XML

```

<NamedResource name="Sample.png" index="77" uri="ms-
resource://SampleApp/Files/Images/Sample.png">
  <Decision index="2">
    <QualifierSet index="1">
      <Qualifier name="Scale" value="180" priority="500"
scoreAsDefault="1.0" index="1"/>
    </QualifierSet>
    <QualifierSet index="2">
      <Qualifier name="Scale" value="140" priority="500"
scoreAsDefault="0.7" index="2"/>
    </QualifierSet>
  </Decision>
  <Candidate type="Path">
    <QualifierSet index="1">
      <Qualifier name="Scale" value="180" priority="500"
scoreAsDefault="1.0" index="1"/>
    </QualifierSet>
    <Value>Images\Sample.scale-180.png</Value>
  </Candidate>
  <Candidate type="Path">
    <QualifierSet index="2">
      <Qualifier name="Scale" value="140" priority="500"
scoreAsDefault="1.0" index="1"/>
    </QualifierSet>
    <Value>Images\Sample.scale-140.png</Value>
  </Candidate>
</NamedResource>

```

ResFiles

The ResFiles indexer is identified by a `type` attribute of RESFILES. It indexes the contents of a `.resfiles` file. Its configuration element conforms to the following schema.

XML

```

<xs:schema id="resx" xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">
  <xs:simpleType name="IndexerConfigResFileType">
    <xs:restriction base="xs:string">
      <xs:pattern value="((r|R)(e|E)(s|S)(f|F)(i|I)(l|L)(e|E)(s|S))"/>
    </xs:restriction>
  </xs:simpleType>

```

```

        </xs:restriction>
    </xs:simpleType>
    <xs:element name="indexer-config">
        <xs:complexType>
            <xs:attribute name="type" type="IndexerConfigResFilesType"
use="required"/>
            <xs:attribute name="qualifierDelimiter" type="xs:string"
use="required"/>
        </xs:complexType>
    </xs:element>
</xs:schema>

```

A `.resfiles` file is a text file containing a flat list of file paths. A `.resfiles` file can contain `"/"` comments. Here's an example.

```

Strings\component1\fr\elements.resjson
Images\logo.scale-100.png
Images\logo.scale-140.png
Images\logo.scale-180.png

```

ResJSON

The ResJSON indexer is identified by a `type` attribute of RESJSON. It indexes the contents of a `.resjson` file, which is a string resource file. Its configuration element conforms to the following schema.

XML

```

<xs:schema id="resjson" xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">
    <xs:simpleType name="IndexerConfigResJsonType">
        <xs:restriction base="xs:string">
            <xs:pattern value="((r|R)(e|E)(s|S)(j|J)(s|S)(o|O)(n|N))"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:element name="indexer-config">
        <xs:complexType>
            <xs:attribute name="type" type="IndexerConfigResJsonType"
use="required"/>
            <xs:attribute name="initialPath" type="xs:string"
use="optional"/>
        </xs:complexType>
    </xs:element>
</xs:schema>

```

A `.resjson` file contains JSON text (see [The application/json Media Type for JavaScript Object Notation \(JSON\)](#)). The file must contain a single JSON object with hierarchical

properties. Each property must either be another JSON object or a string value.

JSON properties with names that begin with an underscore ("_") are not compiled into the final PRI file, but are maintained in the log file.

The file can also contain "/*" comments which are ignored during parsing.

The `initialPath` attribute places all resources under this initial path by prepending it to the name of the resource. You would typically use this when indexing class library resources. The default is blank.

ResW

The ResW indexer is identified by a `type` attribute of RESW. It indexes the contents of a `.resw` file, which is a string resource file. Its configuration element conforms to the following schema.

XML

```
<xs:schema id="resw" xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">
  <xs:simpleType name="IndexerConfigResxType">
    <xs:restriction base="xs:string">
      <xs:pattern value="((r|R)(e|E)(s|S)(w|W))"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:element name="indexer-config">
    <xs:complexType>
      <xs:attribute name="type" type="IndexerConfigResxType"
use="required"/>
      <xs:attribute name="convertDotsToSlashes" type="xs:boolean"
use="required"/>
      <xs:attribute name="initialPath" type="xs:string"
use="optional"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

A `.resw` file is an XML file that conforms to the following schema.

XML

```
<xsd:schema id="root" xmlns=""
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:msdata="urn:schemas-
microsoft-com:xml-msdata">
  <xsd:import namespace="http://www.w3.org/XML/1998/namespace" />
  <xsd:element name="root" msdata:IsDataSet="true">
    <xsd:complexType>
```

```

<xsd:choice maxOccurs="unbounded">
  <xsd:element name="metadata">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="value" type="xsd:string" minOccurs="0" />
      </xsd:sequence>
      <xsd:attribute name="name" use="required" type="xsd:string" />
      <xsd:attribute name="type" type="xsd:string" />
      <xsd:attribute name="mimetype" type="xsd:string" />
      <xsd:attribute ref="xml:space" />
    </xsd:complexType>
  </xsd:element>
  <xsd:element name="assembly">
    <xsd:complexType>
      <xsd:attribute name="alias" type="xsd:string" />
      <xsd:attribute name="name" type="xsd:string" />
    </xsd:complexType>
  </xsd:element>
  <xsd:element name="data">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="value" type="xsd:string" minOccurs="0"
msdata:Ordinal="1" />
        <xsd:element name="comment" type="xsd:string" minOccurs="0"
msdata:Ordinal="2" />
      </xsd:sequence>
      <xsd:attribute name="name" type="xsd:string" use="required"
msdata:Ordinal="1" />
      <xsd:attribute name="type" type="xsd:string"
msdata:Ordinal="3" />
      <xsd:attribute name="mimetype" type="xsd:string"
msdata:Ordinal="4" />
      <xsd:attribute ref="xml:space" />
    </xsd:complexType>
  </xsd:element>
  <xsd:element name="resheader">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="value" type="xsd:string" minOccurs="0"
msdata:Ordinal="1" />
      </xsd:sequence>
      <xsd:attribute name="name" type="xsd:string" use="required" />
    </xsd:complexType>
  </xsd:element>
</xsd:choice>
</xsd:complexType>
</xsd:element>
</xsd:schema>

```

The `convertDotsToSlashes` attribute converts all dot (".") characters found in resource names (data element name attributes) to a forward slash "/", except when the dot characters are between a "[" and "]".

The `initialPath` attribute places all resources under this initial path by prepending it to the name of the resource. You typically use this when indexing class library resources. The default is blank.

Related topics

- [Compile resources manually with MakePri.exe](#)
- [MakePri.exe command-line options](#)
- [MakePri.exe configuration file](#)
- [The application/json Media Type for JavaScript Object Notation \(JSON\) !\[\]\(36f8637baaa56c4be44b454435949289_img.jpg\)](#)

Use the Windows 10 Resource Management System in a legacy app or game

Article • 10/20/2022

.NET and Win32 apps and games are often localized into different languages to expand their total addressable market. For more info about the value proposition of localizing your app, see [Globalization and localization](#). By packaging your .NET or Win32 app or game as an .msix or .appx package, you can leverage the Resource Management System to load app resources tailored to the run-time context. This in-depth topic describes the techniques.

There are many ways to localize a traditional Win32 application, but Windows 8 introduced a [new resource-management system](#) that works across programming languages, across application types, and provides functionality over and above simple localization. This system will be referred to as "MRT" in this topic. Historically, that stood for "Modern Resource Technology" but the term "Modern" has been discontinued. The resource manager might also be known as MRM (Modern Resource Manager) or PRI (Package Resource Index).

Combined with MSIX-based or .appx-based deployment (for example, from the Microsoft Store), MRT can automatically deliver the most-applicable resources for a given user / device which minimizes the download and install size of your application. This size reduction can be significant for applications with a large amount of localized content, perhaps on the order of several *gigabytes* for AAA games. Additional benefits of MRT include localized listings in the Windows Shell and the Microsoft Store, automatic fallback logic when a user's preferred language doesn't match your available resources.

This document describes the high-level architecture of MRT and provides a porting guide to help move legacy Win32 applications to MRT with minimal code changes. Once the move to MRT is made, additional benefits (such as the ability to segment resources by scale factor or system theme) become available to the developer. Note that MRT-based localization works for both UWP applications and Win32 applications processed by the Desktop Bridge (aka "Centennial").

In many situations, you can continue to use your existing localization formats and source code whilst integrating with MRT for resolving resources at runtime and minimizing download sizes - it's not an all-or-nothing approach. The following table summarizes the work and estimated cost/benefit of each stage. This table doesn't

include non-localization tasks, such as providing high-resolution or high-contrast application icons. For more info about providing multiple assets for tiles, icons, etc., See [Tailor your resources for language, scale, high contrast, and other qualifiers](#).

Work	Benefit	Estimated cost
Localize package manifest	Bare minimum work required to have your localized content appear in the Windows Shell and in the Microsoft Store	Small
Use MRT to identify and locate resources	Pre-requisite to minimizing download and install sizes; automatic language fallback	Medium
Build resource packs	Final step to minimize download and install sizes	Small
Migrate to MRT resource formats and APIs	Significantly smaller file sizes (depending on existing resource technology)	Large

Introduction

Most non-trivial applications contain user-interface elements known as *resources* that are decoupled from the application's code (contrasted with *hard-coded values* that are authored in the source code itself). There are several reasons to prefer resources over hard-coded values - ease of editing by non-developers, for example - but one of the key reasons is to enable the application to pick different representations of the same logical resource at runtime. For example, the text to display on a button (or the image to display in an icon) might differ depending on the language(s) the user understands, the characteristics of the display device, or whether the user has any assistive technologies enabled.

Thus the primary purpose of any resource-management technology is to translate, at runtime, a request for a logical or symbolic *resource name* (such as `SAVE_BUTTON_LABEL`) into the best possible actual *value* (eg, "Save") from a set of possible *candidates* (eg, "Save", "Speichern", or "저장"). MRT provides such a function, and enables applications to identify resource candidates using a wide variety of attributes, called *qualifiers*, such as the user's language, the display scale-factor, the user's selected theme, and other environmental factors. MRT even supports custom qualifiers for applications that need it (for example, an application could provide different graphic assets for users that had logged in with an account vs. guest users, without explicitly adding this check into every part of their application). MRT works with both string resources and file-based

resources, where file-based resources are implemented as references to the external data (the files themselves).

Example

Here's a simple example of an application that has text labels on two buttons (`openButton` and `saveButton`) and a PNG file used for a logo (`logoImage`). The text labels are localized into English and German, and the logo is optimized for normal desktop displays (100% scale factor) and high-resolution phones (300% scale factor). Note that this diagram presents a high-level, conceptual view of the model; it does not map exactly to implementation.

Source code

```
openButton.Label = GetResource("OPEN_BUTTON_LABEL");  
saveButton.Label = GetResource("SAVE_BUTTON_LABEL");  
logoImage.Source = GetResource("LOGO_IMAGE_SOURCE");
```

Lookup table

Resource Name	Qualifier / value	Candidate
OPEN_BUTTON_LABEL	Language = EN	Open
OPEN_BUTTON_LABEL	Language = DE	Offen
SAVE_BUTTON_LABEL	Language = EN	Save
SAVE_BUTTON_LABEL	Language = DE	Speichern
LOGO_IMAGE_SOURCE	Scale = 100%	Assets\scale-100\Logo.png
LOGO_IMAGE_SOURCE	Scale = 300%	Assets\scale-300\Logo.png

Files on disk



In the graphic, the application code references the three logical resource names. At runtime, the `GetResource` pseudo-function uses MRT to look those resource names up in the resource table (known as PRI file) and find the most appropriate candidate based on the ambient conditions (the user's language and the display's scale-factor). In the case of the labels, the strings are used directly. In the case of the logo image, the strings are interpreted as filenames and the files are read off disk.

If the user speaks a language other than English or German, or has a display scale-factor other than 100% or 300%, MRT picks the "closest" matching candidate based on a set of fallback rules (see [Resource Management System](#) for more background).

Note that MRT supports resources that are tailored to more than one qualifier - for example, if the logo image contained embedded text that also needed to be localized, the logo would have four candidates: EN/Scale-100, DE/Scale-100, EN/Scale-300 and DE/Scale-300.

Sections in this document

The following sections outline the high-level tasks required to integrate MRT with your application.

Phase 0: Build an application package

This section outlines how to get your existing Desktop application building as an application package. No MRT features are used at this stage.

Phase 1: Localize the application manifest

This section outlines how to localize your application's manifest (so that it appears correctly in the Windows Shell) whilst still using your legacy resource format and API to package and locate resources.

Phase 2: Use MRT to identify and locate resources

This section outlines how to modify your application code (and possibly resource layout) to locate resources using MRT, whilst still using your existing resource formats and APIs to load and consume the resources.

Phase 3: Build resource packs

This section outlines the final changes needed to separate your resources into separate *resource packs*, minimizing the download (and install) size of your app.

Not covered in this document

After completing Phases 0-3 above, you will have an application "bundle" that can be submitted to the Microsoft Store and that will minimize the download and install size for users by omitting the resources they don't need (eg, languages they don't speak). Further improvements in application size and functionality can be made by taking one final step.

Phase 4: Migrate to MRT resource formats and APIs

This phase is beyond the scope of this document; it entails moving your resources (particularly strings) from legacy formats such as MUI DLLs or .NET resource assemblies into PRI files. This can lead to further space savings for download & install sizes. It also allows use of other MRT features such as minimizing the download and install of image files by based on scale factor, accessibility settings, and so on.

Phase 0: Build an application package

Before you make any changes to your application's resources, you must first replace your current packaging and installation technology with the standard UWP packaging and deployment technology. There are three ways to do this:

- If you have a large desktop application with a complex installer or you utilize lots of OS extensibility points, you can use the Desktop App Converter tool to generate the UWP file layout and manifest information from your existing app installer (for example, an MSI).
- If you have a smaller desktop application with relatively few files or a simple installer and no extensibility hooks, you can create the file layout and manifest information manually.
- If you're rebuilding from source and want to update your app to be a pure UWP application, you can create a new project in Visual Studio and rely on the IDE to do much of the work for you.

If you want to use the [Desktop App Converter](#), see [Package a desktop application using the Desktop App Converter](#) for more information on the conversion process. A complete set of Desktop Converter samples can be found on [the Desktop Bridge to UWP samples GitHub repo](#).

If you want to manually create the package, you will need to create a directory structure that includes all your application's files (executables and content, but not source code) and a package manifest file (.appxmanifest). An example can be found in [the Hello, World GitHub sample](#), but a basic package manifest file that runs the desktop executable named `ContosoDemo.exe` is as follows, where the highlighted text would be replaced by your own values.

XML

```
<?xml version="1.0" encoding="utf-8" ?>
<Package
  xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:mp="http://schemas.microsoft.com/appx/2014/phone/manifest"
```

```

xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"

xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows10/restrictedcapabilities"

    IgnorableNamespaces="uap mp rescap">
    <Identity Name="Contoso.Demo"
        Publisher="CN=Contoso.Demo"
        Version="1.0.0.0" />
    <Properties>
    <DisplayName>Contoso App</DisplayName>
    <PublisherDisplayName>Contoso, Inc</PublisherDisplayName>
    <Logo>Assets\StoreLogo.png</Logo>
    </Properties>
    <Dependencies>
    <TargetDeviceFamily Name="Windows.Desktop" MinVersion="10.0.14393.0"
        MaxVersionTested="10.0.14393.0" />
    </Dependencies>
    <Resources>
    <Resource Language="en-US" />
    </Resources>
    <Applications>
    <Application Id="ContosoDemo" Executable="ContosoDemo.exe"
        EntryPoint="Windows.FullTrustApplication">
    <uap:VisualElements DisplayName="Contoso Demo" BackgroundColor="#777777"
        Square150x150Logo="Assets\Square150x150Logo.png"
        Square44x44Logo="Assets\Square44x44Logo.png"
        Description="Contoso Demo">
    </uap:VisualElements>
    </Application>
    </Applications>
    <Capabilities>
    <rescap:Capability Name="runFullTrust" />
    </Capabilities>
    </Package>

```

For more information about the package manifest file and package layout, see [App package manifest](#).

Finally, if you're using Visual Studio to create a new project and migrate your existing code across, see [Create a "Hello, world" app](#). You can include your existing code into the new project, but you will likely have to make significant code changes (particularly in the user interface) in order to run as a pure UWP app. These changes are outside the scope of this document.

Phase 1: Localize the manifest

Step 1.1: Update strings & assets in the manifest

In Phase 0 you created a basic package manifest (.appxmanifest) file for your application (based on values provided to the converter, extracted from the MSI, or manually entered into the manifest) but it will not contain localized information, nor will it support additional features like high-resolution Start tile assets, etc.

To ensure your application's name and description are correctly localized, you must define some resources in a set of resource files, and update the package manifest to reference them.

Creating a default resource file

The first step is to create a default resource file in your default language (eg, US English). You can do this either manually with a text editor, or via the Resource Designer in Visual Studio.

If you want to create the resources manually:

1. Create an XML file named `resources.resw` and place it in a `Strings\en-us` subfolder of your project. Use the appropriate BCP-47 code if your default language is not US English.
2. In the XML file, add the following content, where the highlighted text is replaced with the appropriate text for your app, in your default language.

ⓘ Note

There are restrictions on the lengths of some of these strings. For more info, see [VisualElements](#).

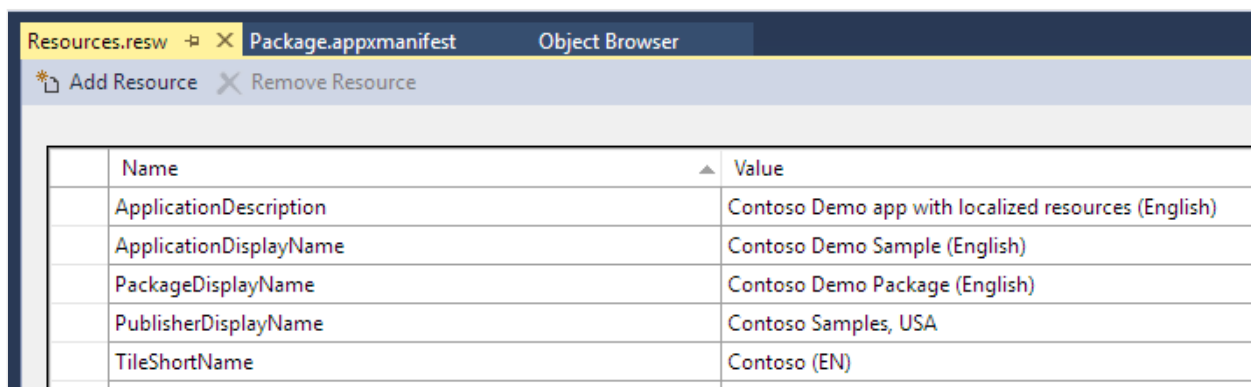
XML

```
<?xml version="1.0" encoding="utf-8"?>
<root>
  <data name="ApplicationDescription">
    <value>Contoso Demo app with localized resources (English)</value>
  </data>
  <data name="ApplicationDisplayName">
    <value>Contoso Demo Sample (English)</value>
  </data>
  <data name="PackageDisplayName">
    <value>Contoso Demo Package (English)</value>
  </data>
  <data name="PublisherDisplayName">
    <value>Contoso Samples, USA</value>
  </data>
  <data name="TileShortName">
```

```
<value>Contoso (EN)</value>
</data>
</root>
```

If you want to use the designer in Visual Studio:

1. Create the `Strings\en-us` folder (or other language as appropriate) in your project and add a **New Item** to the root folder of your project, using the default name of `resources.resw`. Be sure to choose **Resources File (.resw)** and not **Resource Dictionary** - a Resource Dictionary is a file used by XAML applications.
2. Using the designer, enter the following strings (use the same `Names` but replace the `Values` with the appropriate text for your application):



The screenshot shows the Visual Studio interface with the `Resources.resw` file open in the designer. The table below represents the data shown in the designer.

Name	Value
ApplicationDescription	Contoso Demo app with localized resources (English)
ApplicationDisplayName	Contoso Demo Sample (English)
PackageDisplayName	Contoso Demo Package (English)
PublisherDisplayName	Contoso Samples, USA
TileShortName	Contoso (EN)

ⓘ Note

If you start with the Visual Studio designer, you can always edit the XML directly by pressing `F7`. But if you start with a minimal XML file, *the designer will not recognize the file* because it's missing a lot of additional metadata; you can fix this by copying the boilerplate XSD information from a designer-generated file into your hand-edited XML file.

Update the manifest to reference the resources

After you have the values defined in the `.resw` file, the next step is to update the manifest to reference the resource strings. Again, you can edit an XML file directly, or rely on the Visual Studio Manifest Designer.

If you are editing XML directly, open the `AppxManifest.xml` file and make the following changes to the highlighted values - use this *exact* text, not text specific to your application. There is no requirement to use these exact resource names—you can choose your own—but whatever you choose must exactly match whatever is in the

.resw file. These names should match the `Names` you created in the .resw file, prefixed with the `ms-resource:` scheme and the `Resources/` namespace.

ⓘ Note

Many elements of the manifest have been omitted from this snippet - do not delete anything!

XML

```
<?xml version="1.0" encoding="utf-8"?>
<Package>
  <Properties>
    <DisplayName>ms-resource:Resources/PackageDisplayName</DisplayName>
    <PublisherDisplayName>ms-
resource:Resources/PublisherDisplayName</PublisherDisplayName>
  </Properties>
  <Applications>
    <Application>
      <uap:VisualElements DisplayName="ms-
resource:Resources/ApplicationDisplayName"
        Description="ms-resource:Resources/ApplicationDescription">
        <uap:DefaultTile ShortName="ms-resource:Resources/TileShortName">
          <uap:ShowNameOnTiles>
            <uap:ShowOn Tile="square150x150Logo" />
          </uap:ShowNameOnTiles>
        </uap:DefaultTile>
      </uap:VisualElements>
    </Application>
  </Applications>
</Package>
```

If you are using the Visual Studio manifest designer, open the .appxmanifest file and change the highlighted values values in the **Application* tab and the *Packaging* tab:

Package.appxmanifest Object Browser

The properties of the deployment package for your app are contained in the app manifest file. You can use the Manifest Designer to set or modify

Application Visual Assets Capabilities Declarations Content URIs Packaging

Use this page to set the properties that identify and describe your app.

Display name:

Entry point:

Default language: [More information](#)

Description:

Supported rotations: An optional setting that indicates the app's orientation preferences.

☐ Landscape ☐ Portrait ☐ Landscape-flipped ☐ Portrait-flipped

Package.appxmanifest Object Browser

The properties of the deployment package for your app are contained in the app manifest file. You can use the Manifest Designer to set or modify

Application Visual Assets Capabilities Declarations Content URIs Packaging

Use this page to set the properties that identify and describe your package when it is deployed.

Package name:

Package display name:

Version: Major: Minor: Build: [More information](#)

Publisher:

Publisher display name:

Package family name:

Step 1.2: Build PRI file, make an MSIX package, and verify it's working

You should now be able to build the `.pri` file and deploy the application to verify that the correct information (in your default language) is appearing in the Start Menu.

If you're building in Visual Studio, simply press `Ctrl+Shift+B` to build the project and then right-click on the project and choose `Deploy` from the context menu.

If you're building manually, follow these steps to create a configuration file for `MakePRI` tool and to generate the `.pri` file itself (more information can be found in [Manual app packaging](#)):

1. Open a developer command prompt from the **Visual Studio 2017** or **Visual Studio 2019** folder in the Start menu.
2. Switch to the project root directory (the one that contains the .appxmanifest file and the **Strings** folder).
3. Type the following command, replacing "contoso_demo.xml" with a name suitable for your project, and "en-US" with the default language of your app (or keep it en-US if applicable). Note the XML file is created in the parent directory (**not** in the project directory) since it's not part of the application (you can choose any other directory you want, but be sure to substitute that in future commands).

CMD

```
makepri createconfig /cf ..\contoso_demo.xml /dq en-US /pv 10.0 /o
```

You can type `makepri createconfig /?` to see what each parameter does, but in summary:

- `/cf` sets the Configuration Filename (the output of this command)
- `/dq` sets the Default Qualifiers, in this case the language `en-US`
- `/pv` sets the Platform Version, in this case Windows 10
- `/o` sets it to Overwrite the output file if it exists

4. Now you have a configuration file, run `MakePRI` again to actually search the disk for resources and package them into a PRI file. Replace "contoso_demo.xml" with the XML filename you used in the previous step, and be sure to specify the parent directory for both input and output:

CMD

```
makepri new /pr . /cf ..\contoso_demo.xml /of ..\resources.pri /mf AppX /o
```

You can type `makepri new /?` to see what each parameter does, but in a nutshell:

- `/pr` sets the Project Root (in this case, the current directory)
- `/cf` sets the Configuration Filename, created in the previous step
- `/of` sets the Output File
- `/mf` creates a Mapping File (so we can exclude files in the package in a later step)
- `/o` sets it to Overwrite the output file if it exists

5. Now you have a `.pri` file with the default language resources (eg, en-US). To verify that it worked correctly, you can run the following command:

CMD

```
makepri dump /if ..\resources.pri /of ..\resources /o
```

You can type `makepri dump /?` to see what each parameter does, but in a nutshell:

- `/if` sets the Input Filename
- `/of` sets the Output Filename (`.xml` will be appended automatically)
- `/o` sets it to Overwrite the output file if it exists

6. Finally, you can open `..\resources.xml` in a text editor and verify it lists your `<NamedResource>` values (like `ApplicationDescription` and `PublisherDisplayName`) along with `<Candidate>` values for your chosen default language (there will be other content in the beginning of the file; ignore that for now).

You can open the mapping file `..\resources.map.txt` to verify it contains the files needed for your project (including the PRI file, which is not part of the project's directory). Importantly, the mapping file will *not* include a reference to your `resources.resw` file because the contents of that file have already been embedded in the PRI file. It will, however, contain other resources like the filenames of your images.

Building and signing the package

Now the PRI file is built, you can build and sign the package:

1. To create the app package, run the following command replacing `contoso_demo.appx` with the name of the `.msix/.appx` file you want to create and making sure to choose a different directory for the file (this sample uses the parent directory; it can be anywhere but should **not** be the project directory).

CMD

```
makeappx pack /m AppXManifest.xml /f ..\resources.map.txt /p  
..\contoso_demo.appx /o
```

You can type `makeappx pack /?` to see what each parameter does, but in a nutshell:

- `/m` sets the Manifest file to use
- `/f` sets the mapping File to use (created in the previous step)

- `/p` sets the output Package name
- `/o` sets it to Overwrite the output file if it exists

2. After the package is created, it must be signed. The easiest way to get a signing certificate is by creating an empty Universal Windows project in Visual Studio and copying the `.pfx` file it creates, but you can create one manually using the `MakeCert` and `Pvk2Pfx` utilities as described in [How to create an app package signing certificate](#).

Important

If you manually create a signing certificate, make sure you place the files in a different directory than your source project or your package source, otherwise it might get included as part of the package, including the private key!

3. To sign the package, use the following command. Note that the `Publisher` specified in the `Identity` element of the `AppxManifest.xml` must match the `Subject` of the certificate (this is **not** the `<PublisherDisplayName>` element, which is the localized display name to show to users). As usual, replace the `contoso_demo...` filenames with the names appropriate for your project, and (**very important**) make sure the `.pfx` file is not in the current directory (otherwise it would have been created as part of your package, including the private signing key!):

CMD

```
signtool sign /fd SHA256 /a /f ..\contoso_demo_key.pfx  
..\contoso_demo.appx
```

You can type `signtool sign /?` to see what each parameter does, but in a nutshell:

- `/fd` sets the File Digest algorithm (SHA256 is the default for `.appx`)
- `/a` will Automatically select the best certificate
- `/f` specifies the input File that contains the signing certificate

Finally, you can now double-click on the `.appx` file to install it, or if you prefer the command-line you can open a PowerShell prompt, change to the directory containing the package, and type the following (replacing `contoso_demo.appx` with your package name):

CMD

```
add-appxpackage contoso_demo.appx
```

If you receive errors about the certificate not being trusted, make sure it is added to the machine store (**not** the user store). To add the certificate to the machine store, you can either use the command-line or Windows Explorer.

To use the command-line:

1. Run a Visual Studio 2017 or Visual Studio 2019 command prompt as an Administrator.
2. Switch to the directory that contains the `.cer` file (remember to ensure this is outside of your source or project directories!)
3. Type the following command, replacing `contoso_demo.cer` with your filename:

CMD

```
certutil -addstore TrustedPeople contoso_demo.cer
```

You can run `certutil -addstore /?` to see what each parameter does, but in a nutshell:

- `-addstore` adds a certificate to a certificate store
- `TrustedPeople` indicates the store into which the certificate is placed

To use Windows Explorer:

1. Navigate to the folder that contains the `.pfx` file
2. Double-click on the `.pfx` file and the **Certificate Import Wizard** should appear
3. Choose `Local Machine` and click `Next`
4. Accept the User Account Control admin elevation prompt, if it appears, and click `Next`
5. Enter the password for the private key, if there is one, and click `Next`
6. Select `Place all certificates in the following store`
7. Click `Browse`, and choose the `Trusted People` folder (**not** "Trusted Publishers")
8. Click `Next` and then `Finish`

After adding the certificate to the `Trusted People` store, try installing the package again.

You should now see your app appear in the Start Menu's "All Apps" list, with the correct information from the `.resw` / `.pri` file. If you see a blank string or the string `ms-`

`resource:...` then something has gone wrong - double check your edits and make sure they're correct. If you right-click on your app in the Start Menu, you can Pin it as a tile and verify the correct information is displayed there also.

Step 1.3: Add more supported languages

After the changes have been made to the package manifest and the initial `resources.resw` file has been created, adding additional languages is easy.

Create additional localized resources

First, create the additional localized resource values.

Within the `Strings` folder, create additional folders for each language you support using the appropriate BCP-47 code (for example, `Strings\de-DE`). Within each of these folders, create a `resources.resw` file (using either an XML editor or the Visual Studio designer) that includes the translated resource values. It is assumed you already have the localized strings available somewhere, and you just need to copy them into the `.resw` file; this document does not cover the translation step itself.

For example, the `Strings\de-DE\resources.resw` file might look like this, with the highlighted text changed from `en-US`:

XML

```
<?xml version="1.0" encoding="utf-8"?>
<root>
  <data name="ApplicationDescription">
    <value>Contoso Demo app with localized resources (German)</value>
  </data>
  <data name="ApplicationDisplayName">
    <value>Contoso Demo Sample (German)</value>
  </data>
  <data name="PackageDisplayName">
    <value>Contoso Demo Package (German)</value>
  </data>
  <data name="PublisherDisplayName">
    <value>Contoso Samples, DE</value>
  </data>
  <data name="TileShortName">
    <value>Contoso (DE)</value>
  </data>
</root>
```

The following steps assume you added resources for both `de-DE` and `fr-FR`, but the same pattern can be followed for any language.

Update the package manifest to list supported languages

The package manifest must be updated to list the languages supported by the app. The Desktop App Converter adds the default language, but the others must be added explicitly. If you're editing the `AppxManifest.xml` file directly, update the `Resources` node as follows, adding as many elements as you need, and substituting the appropriate languages you support and making sure the first entry in the list is the default (fallback) language. In this example, the default is English (US) with additional support for both German (Germany) and French (France):

XML

```
<Resources>
  <Resource Language="EN-US" />
  <Resource Language="DE-DE" />
  <Resource Language="FR-FR" />
</Resources>
```

If you are using Visual Studio, you shouldn't need to do anything; if you look at `Package.appxmanifest` you should see the special `x-generate` value, which causes the build process to insert the languages it finds in your project (based on the folders named with BCP-47 codes). Note that this is not a valid value for a real package manifest; it only works for Visual Studio projects:

XML

```
<Resources>
  <Resource Language="x-generate" />
</Resources>
```

Re-build with the localized values

Now you can build and deploy your application, again, and if you change your language preference in Windows you should see the newly-localized values appear in the Start menu (instructions for how to change your language are below).

For Visual Studio, again you can just use `Ctrl+Shift+B` to build, and right-click the project to `Deploy`.

If you're manually building the project, follow the same steps as above but add the additional languages, separated by underscores, to the default qualifiers list (/dq) when creating the configuration file. For example, to support the English, German, and French resources added in the previous step:

CMD

```
makepri createconfig /cf ..\contoso_demo.xml /dq en-US_de-DE_fr-FR /pv 10.0 /o
```

This will create a PRI file that contains all the specified languages that you can easily use for testing. If the total size of your resources is small, or you only support a small number of languages, this might be acceptable for your shipping app; it's only if you want the benefits of minimizing install / download size for your resources that you need to do the additional work of building separate language packs.

Test with the localized values

To test the new localized changes, you simply add a new preferred UI language to Windows. There is no need to download language packs, reboot the system, or have your entire Windows UI appear in a foreign language.

1. Run the **Settings** app (Windows + I)
2. Go to **Time & language**
3. Go to **Region & language**
4. Click **Add a language**
5. Type (or select) the language you want (eg **Deutsch** or **German**)
 - If there are sub-languages, choose the one you want (eg, **Deutsch / Deutschland**)
6. Select the new language in the language list
7. Click **Set as default**

Now open the Start menu and search for your application, and you should see the localized values for the selected language (other apps might also appear localized). If you don't see the localized name right away, wait a few minutes until the Start Menu's cache is refreshed. To return to your native language, just make it the default language in the language list.

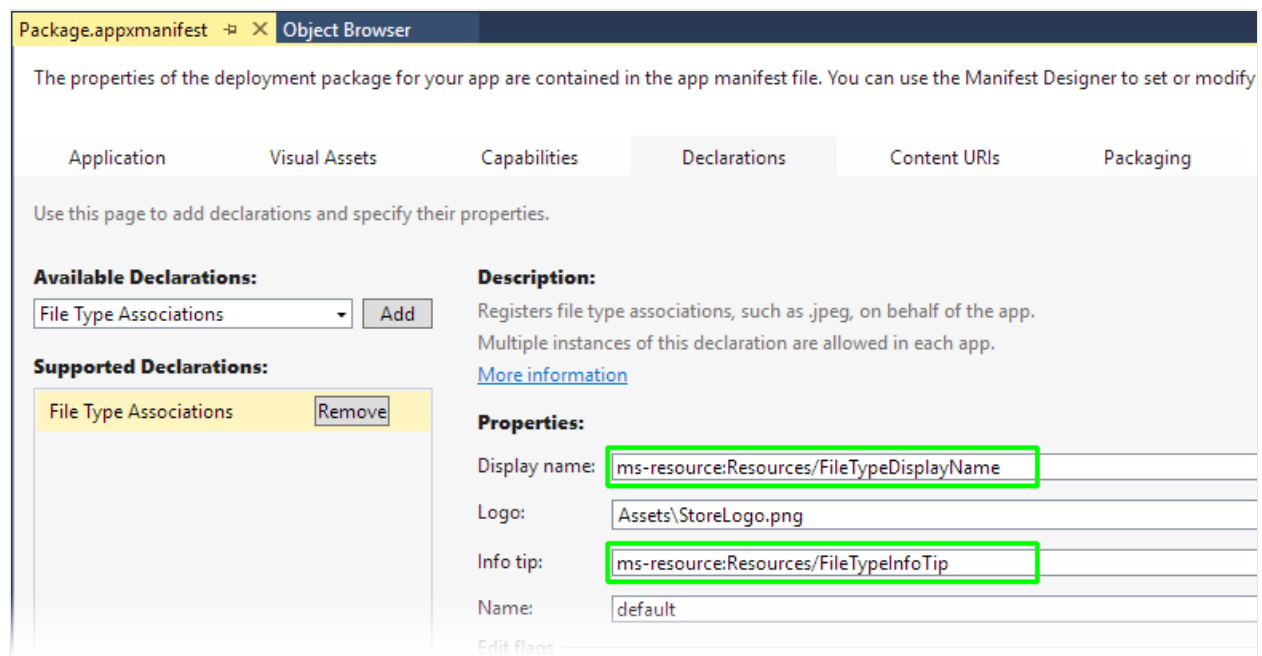
Step 1.4: Localizing more parts of the package manifest (optional)

Other sections of the package manifest can be localized. For example, if your application handles file-extensions then it should have a `windows.fileTypeAssociation` extension in the manifest, using the green highlighted text exactly as shown (since it will refer to resources), and replacing the yellow highlighted text with information specific to your application:

```
XML

<Extensions>
  <uap:Extension Category="windows.fileTypeAssociation">
    <uap:FileTypeAssociation Name="default">
      <uap:DisplayName>ms-
resource:Resources/FileTypeDisplayName</uap:DisplayName>
      <uap:Logo>Assets\StoreLogo.png</uap:Logo>
      <uap:InfoTip>ms-resource:Resources/FileTypeInfoTip</uap:InfoTip>
      <uap:SupportedFileTypes>
        <uap:FileType ContentType="application/x-
contoso">.contoso</uap:FileType>
      </uap:SupportedFileTypes>
    </uap:FileTypeAssociation>
  </uap:Extension>
</Extensions>
```

You can also add this information using the Visual Studio Manifest Designer, using the `Declarations` tab, taking note of the highlighted values:

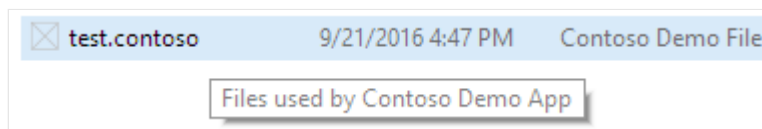


Now add the corresponding resource names to each of your `.resw` files, replacing the highlighted text with the appropriate text for your app (remember to do this for *each supported language!*):

```
XML
```

```
... existing content...  
<data name="FileTypeDisplayName">  
  <value>Contoso Demo File</value>  
</data>  
<data name="FileTypeInfoTip">  
  <value>Files used by Contoso Demo App</value>  
</data>
```

This will then show up in parts of the Windows shell, such as File Explorer:



Build and test the package as before, exercising any new scenarios that should show the new UI strings.

Phase 2: Use MRT to identify and locate resources

The previous section showed how to use MRT to localize your app's manifest file so that the Windows Shell can correctly display the app's name and other metadata. No code changes were required for this; it simply required the use of `.resw` files and some additional tools. This section will show how to use MRT to locate resources in your existing resource formats and using your existing resource-handling code with minimal changes.

Assumptions about existing file layout & application code

Because there are many ways to localize Win32 Desktop apps, this paper will make some simplifying assumptions about the existing application's structure that you will need to map to your specific environment. You might need to make some changes to your existing codebase or resource layout to conform to the requirements of MRT, and those are largely out of scope for this document.

Resource file layout

This article assumes your localized resources all have the same filenames (eg, `contoso_demo.exe.mui` or `contoso_strings.dll` or `contoso.strings.xml`) but that they are placed in different folders with BCP-47 names (`en-US`, `de-DE`, etc.). It doesn't matter how many resource files you have, what their names are, what their file-formats /

associated APIs are, etc. The only thing that matters is that every *logical* resource has the same filename (but placed in a different *physical* directory).

As a counter-example, if your application uses a flat file-structure with a single `Resources` directory containing the files `english_strings.dll` and `french_strings.dll`, it would not map well to MRT. A better structure would be a `Resources` directory with subdirectories and files `en\strings.dll` and `fr\strings.dll`. It's also possible to use the same base filename but with embedded qualifiers, such as `strings.lang-en.dll` and `strings.lang-fr.dll`, but using directories with the language codes is conceptually simpler so it's what we'll focus on.

ⓘ Note

It is still possible to use MRT and the benefits of packaging even if you can't follow this file naming convention; it just requires more work.

For example, the application might have a set of custom UI commands (used for button labels etc.) in a simple text file named `ui.txt`, laid out under a `UICommands` folder:

```
+ ProjectRoot
|--+ Strings
|   |--+ en-US
|       \--- resources.resw
|   \---+ de-DE
|           \--- resources.resw
|--+ UICommands
|   |--+ en-US
|       \--- ui.txt
|   \---+ de-DE
|           \--- ui.txt
|--- AppxManifest.xml
|--- ...rest of project...
```

Resource loading code

This article assumes that at some point in your code you want to locate the file that contains a localized resource, load it, and then use it. The APIs used to load the resources, the APIs used to extract the resources, etc. are not important. In pseudocode, there are basically three steps:

```
set userLanguage = GetUsersPreferredLanguage()
set resourceFile = FindResourceFileForLanguage(MY_RESOURCE_NAME,
userLanguage)
```

```
set resource = LoadResource(resourceFile)

// now use 'resource' however you want
```

MRT only requires changing the first two steps in this process - how you determine the best candidate resources and how you locate them. It doesn't require you to change how you load or use the resources (although it provides facilities for doing that if you want to take advantage of them).

For example, the application might use the Win32 API `GetUserPreferredUILanguages`, the CRT function `sprintf`, and the Win32 API `CreateFile` to replace the three pseudocode functions above, then manually parse the text file looking for `name=value` pairs. (The details are not important; this is merely to illustrate that MRT has no impact on the techniques used to handle resources once they have been located).

Step 2.1: Code changes to use MRT to locate files

Switching your code to use MRT for locating resources is not difficult. It requires using a handful of WinRT types and a few lines of code. The main types that you will use are as follows:

- [ResourceContext](#), which encapsulates the currently active set of qualifier values (language, scale factor, etc.)
- [ResourceManager](#) (the WinRT version, not the .NET version), which enables access to all the resources from the PRI file
- [ResourceMap](#), which represents a specific subset of the resources in the PRI file (in this example, the file-based resources vs. the string resources)
- [NamedResource](#), which represents a logical resource and all its possible candidates
- [ResourceCandidate](#), which represents a single concrete candidate resource

In pseudo-code, the way you would resolve a given resource file name (like `UICommands\ui.txt` in the sample above) is as follows:

```
// Get the ResourceContext that applies to this app
set resourceContext = ResourceContext.GetForViewIndependentUse()

// Get the current ResourceManager (there's one per app)
set resourceManager = ResourceManager.Current

// Get the "Files" ResourceMap from the ResourceManager
set fileResources = resourceManager.MainResourceMap.GetSubtree("Files")

// Find the NamedResource with the logical filename we're looking for,
// by indexing into the ResourceMap
```

```
set desiredResource = fileResources["UICommands\ui.txt"]

// Get the ResourceCandidate that best matches our ResourceContext
set bestCandidate = desiredResource.Resolve(resourceContext)

// Get the string value (the filename) from the ResourceCandidate
set absoluteFileName = bestCandidate.ValueAsString
```

Note in particular that the code does **not** request a specific language folder - like `UICommands\en-US\ui.txt` - even though that is how the files exist on-disk. Instead, it asks for the *logical* filename `UICommands\ui.txt` and relies on MRT to find the appropriate on-disk file in one of the language directories.

From here, the sample app could continue to use `CreateFile` to load the `absoluteFileName` and parse the `name=value` pairs just as before; none of that logic needs to change in the app. If you are writing in C# or C++/CX, the actual code is not much more complicated than this (and in fact many of the intermediate variables can be elided) - see the section on **Loading .NET resources**, below. C++/WRL-based applications will be more complex due to the low-level COM-based APIs used to activate and call the WinRT APIs, but the fundamental steps you take are the same - see the section on **Loading Win32 MUI resources**, below.

Loading .NET resources

Because .NET has a built-in mechanism for locating and loading resources (known as "Satellite Assemblies"), there is no explicit code to replace as in the synthetic example above - in .NET you just need your resource DLLs in the appropriate directories and they are automatically located for you. When an app is packaged as an MSIX or .appx using resource packs, the directory structure is somewhat different - rather than having the resource directories be subdirectories of the main application directory, they are peers of it (or not present at all if the user doesn't have the language listed in their preferences).

For example, imagine a .NET application with the following layout, where all the files exist under the `MainApp` folder:

```
+ MainApp
|--+ en-us
|   \--- MainApp.resources.dll
|--+ de-de
|   \--- MainApp.resources.dll
|--+ fr-fr
```

```
| \--- MainApp.resources.dll
| \--- MainApp.exe
```

After conversion to .appx, the layout will look something like this, assuming `en-US` was the default language and the user has both German and French listed in their language list:

```
+ WindowsAppsRoot
|--+ MainApp_neutral
|  |--+ en-us
|  |  \--- MainApp.resources.dll
|  |  \--- MainApp.exe
|--+ MainApp_neutral_resources.language_de
|  |--+ de-de
|  |  \--- MainApp.resources.dll
|--+ MainApp_neutral_resources.language_fr
|  |--+ fr-fr
|  |  \--- MainApp.resources.dll
```

Because the localized resources no longer exist in sub-directories underneath the main executable's install location, the built-in .NET resource resolution fails. Luckily, .NET has a well-defined mechanism for handling failed assembly load attempts - the `AssemblyResolve` event. A .NET app using MRT must register for this event and provide the missing assembly for the .NET resource subsystem.

A concise example of how to use the WinRT APIs to locate satellite assemblies used by .NET is as follows; the code as-presented is intentionally compressed to show a minimal implementation, although you can see it maps closely to the pseudo-code above, with the passed-in `ResolveEventArgs` providing the name of the assembly we need to locate. A runnable version of this code (with detailed comments and error-handling) can be found in the file `PriResourceResolver.cs` in [the .NET Assembly Resolver sample on GitHub](#).

C#

```
static class PriResourceResolver
{
    internal static Assembly ResolveResourceDll(object sender,
        ResolveEventArgs args)
    {
        var fullAssemblyName = new AssemblyName(args.Name);
        var fileName = string.Format(@"{0}.dll", fullAssemblyName.Name);

        var resourceContext = ResourceContext.GetForViewIndependentUse();
        resourceContext.Languages = new[] { fullAssemblyName.CultureName };
    }
}
```

```

var resource =
ResourceManager.Current.MainResourceMap.GetSubtree("Files")[fileName];

    // Note use of 'UnsafeLoadFrom' - this is required for apps installed
    with .appx, but
    // in general is discouraged. The full sample provides a safer wrapper
    of this method
    return
Assembly.UnsafeLoadFrom(resource.Resolve(resourceContext).ValueAsString);
}
}

```

Given the class above, you would add the following somewhere early-on in your application's startup code (before any localized resources would need to load):

```

C#

void EnableMrtResourceLookup()
{
    AppDomain.CurrentDomain.AssemblyResolve +=
PriResourceResolver.ResolveResourceDll;
}

```

The .NET runtime will raise the `AssemblyResolve` event whenever it can't find the resource DLLs, at which point the provided event handler will locate the desired file via MRT and return the assembly.

ⓘ Note

If your app already has an `AssemblyResolve` handler for other purposes, you will need to integrate the resource-resolving code with your existing code.

Loading Win32 MUI resources

Loading Win32 MUI resources is essentially the same as loading .NET Satellite Assemblies, but using either C++/CX or C++/WRL code instead. Using C++/CX allows for much simpler code that closely matches the C# code above, but it uses C++ language extensions, compiler switches, and additional runtime overhead you might wish to avoid. If that is the case, using C++/WRL provides a much lower-impact solution at the cost of more verbose code. Nevertheless, if you are familiar with ATL programming (or COM in general) then WRL should feel familiar.

The following sample function shows how to use C++/WRL to load a specific resource DLL and return an `HINSTANCE` that can be used to load further resources using the usual

Win32 resource APIs. Note that unlike the C# sample that explicitly initializes the `ResourceContext` with the language requested by the .NET runtime, this code relies on the user's current language.

C++

```
#include <roapi.h>
#include <wrl\client.h>
#include <wrl\wrappers\corewrappers.h>
#include <Windows.ApplicationModel.resources.core.h>
#include <Windows.Foundation.h>

#define IF_FAIL_RETURN(hr) if (FAILED((hr))) return hr;

HRESULT GetMrtResourceHandle(LPCWSTR resourceFilePath, HINSTANCE*
resourceHandle)
{
    using namespace Microsoft::WRL;
    using namespace Microsoft::WRL::Wrappers;
    using namespace ABI::Windows::ApplicationModel::Resources::Core;
    using namespace ABI::Windows::Foundation;

    *resourceHandle = nullptr;
    HRESULT hr{ S_OK };
    RoInitializeWrapper roInit{ RO_INIT_SINGLETHREADED };
    IF_FAIL_RETURN(roInit);

    // Get Windows.ApplicationModel.Resources.Core.ResourceManager statics
    ComPtr<IResourceManagerStatics> resourceManagerStatics;
    IF_FAIL_RETURN(GetActivationFactory(
        HStringReference(
            RuntimeClass_Windows_ApplicationModel_Resources_Core_ResourceManager).Get(),
            &resourceManagerStatics));

    // Get .Current property
    ComPtr<IResourceManager> resourceManager;
    IF_FAIL_RETURN(resourceManagerStatics->get_Current(&resourceManager));

    // get .MainResourceMap property
    ComPtr<IResourceMap> resourceMap;
    IF_FAIL_RETURN(resourceManager->get_MainResourceMap(&resourceMap));

    // Call .GetValue with supplied filename
    ComPtr<IResourceCandidate> resourceCandidate;
    IF_FAIL_RETURN(resourceMap->GetValue(HStringReference(resourceFilePath).Get(),
        &resourceCandidate));

    // Get .ValueAsString property
    HString resolvedResourceFilePath;
    IF_FAIL_RETURN(resourceCandidate->get_ValueAsString(
        resolvedResourceFilePath.GetAddressOf()));
```

```
// Finally, load the DLL and return the hInst.  
*resourceHandle =  
LoadLibraryEx(resolvedResourceFilePath.GetRawBuffer(nullptr),  
    nullptr, LOAD_LIBRARY_AS_DATAFILE | LOAD_LIBRARY_AS_IMAGE_RESOURCE);  
  
return S_OK;  
}
```

Phase 3: Building resource packs

Now that you have a "fat pack" that contains all resources, there are two paths towards building separate main package and resource packages in order to minimize download and install sizes:

- Take an existing fat pack and run it through [the Bundle Generator tool](#) to automatically create resource packs. This is the preferred approach if you have a build system that already produces a fat pack and you want to post-process it to generate the resource packs.
- Directly produce the individual resource packages and build them into a bundle. This is the preferred approach if you have more control over your build system and can build the packages directly.

Step 3.1: Creating the bundle

Using the Bundle Generator tool

In order to use the Bundle Generator tool, the PRI config file created for the package needs to be manually updated to remove the `<packaging>` section.

If you're using Visual Studio, refer to [Ensure that resources are installed on a device regardless of whether a device requires them](#) for information on how to build all languages into the main package by creating the files `priconfig.packaging.xml` and `priconfig.default.xml`.

If you're manually editing files, follow these steps:

1. Create the config file the same way as before, substituting the correct path, file name and languages:

```
CMD
```

```
makepri createconfig /cf ..\contoso_demo.xml /dq en-US_de-DE_es-MX /pv 10.0 /o
```

2. Manually open the created `.xml` file and delete the entire `<packaging>` section (but keep everything else intact):

XML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<resources targetOsVersion="10.0.0" majorVersion="1">
  <!-- Packaging section has been deleted... -->
  <index root="" startIndexAt="">
    <default>
      ...
    </default>
  </index>

```

3. Build the `.pri` file and the `.appx` package as before, using the updated configuration file and the appropriate directory and file names (see above for more information on these commands):

CMD

```
makepri new /pr . /cf ..\contoso_demo.xml /of ..\resources.pri /mf AppX /o
makeappx pack /m AppXManifest.xml /f ..\resources.map.txt /p ..\contoso_demo.appx /o
```

4. After the package has been created, use the following command to create the bundle, using the appropriate directory and file names:

CMD

```
BundleGenerator.exe -Package ..\contoso_demo.appx -Destination ..\bundle -BundleName contoso_demo
```

Now you can move to the final step, signing (see below).

Manually creating resource packages

Manually creating resource packages requires running a slightly different set of commands to build separate `.pri` and `.appx` files - these are all similar to the commands used above to create fat packages, so minimal explanation is given. Note: All the commands assume that the current directory is the directory containing the

AppXManifest.xml file, but all files are placed into the parent directory (you can use a different directory, if necessary, but you shouldn't pollute the project directory with any of these files). As always, replace the "Contoso" filenames with your own file names.

1. Use the following command to create a config file that names **only** the default language as the default qualifier - in this case, `en-US`:

CMD

```
makepri createconfig /cf ..\contoso_demo.xml /dq en-US /pv 10.0 /o
```

2. Create a default `.pri` and `.map.txt` file for the main package, plus an additional set of files for each language found in your project, with the following command:

CMD

```
makepri new /pr . /cf ..\contoso_demo.xml /of ..\resources.pri /mf AppX /o
```

3. Use the following command to create the main package (which contains the executable code and default language resources). As always, change the name as you see fit, although you should put the package in a separate directory to make creating the bundle easier later (this example uses the `..\bundle` directory):

CMD

```
makeappx pack /m .\AppXManifest.xml /f ..\resources.map.txt /p ..\bundle\contoso_demo.main.appx /o
```

4. After the main package has been created, use the following command once for each additional language (ie, repeat this command for each language map file generated in the previous step). Again, the output should be in a separate directory (the same one as the main package). Note the language is specified **both** in the `/f` option and the `/p` option, and the use of the new `/r` argument (which indicates a Resource Package is desired):

CMD

```
makeappx pack /r /m .\AppXManifest.xml /f ..\resources.language-de.map.txt /p ..\bundle\contoso_demo.de.appx /o
```

5. Combine all the packages from the bundle directory into a single `.appxbundle` file. The new `/d` option specifies the directory to use for all the files in the bundle (this

is why the `.appx` files are put into a separate directory in the previous step):

CMD

```
makeappx bundle /d ..\bundle /p ..\contoso_demo.appxbundle /o
```

The final step to building the package is signing.

Step 3.2: Signing the bundle

Once you have created the `.appxbundle` file (either through the Bundle Generator tool or manually) you will have a single file that contains the main package plus all the resource packages. The final step is to sign the file so that Windows will install it:

CMD

```
signtool sign /fd SHA256 /a /f ..\contoso_demo_key.pfx  
..\contoso_demo.appxbundle
```

This will produce a signed `.appxbundle` file that contains the main package plus all the language-specific resource packages. It can be double-clicked just like a package file to install the app plus any appropriate language(s) based on the user's Windows language preferences.

Related topics

- [Tailor your resources for language, scale, high contrast, and other qualifiers](#)

Data access

Article • 09/15/2023

You can store data on the user's device by using a SQLite database. You can also connect your app directly to a SQL Server, MongoDB, or MySQL database without having to use any sort of service layer.

 Expand table

Topic	Description
Use a SQLite database in a UWP app	Shows you how to use SQLite to store and retrieve data in a light-weight database on the users device. SQLite is a server-less, embedded database engine.
Use a SQL server database in a UWP app	Shows you how to connect directly to a SQL Server database and then store and retrieve data by using classes in the System.Data.SqlClient namespace. No service layer required.
Use a MongoDB database in a Windows app	Shows you how to work with a MongoDB database and test the connection programmatically. MongoDB is a cross-platform document database.
Use a MySQL database in a Windows app	Shows you how to connect to a MySQL database and interact with the database in a Windows application. MySQL is an open source, cross-platform relational database.
Use a Cosmos DB database in a Windows app	Shows you how to work with a Cosmos DB database and test the connection programmatically. Cosmos DB is a cloud-based document database.

Related topics

- [Customer Orders Database sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Use a SQLite database in a UWP app

Article • 10/20/2022

You can use SQLite to store and retrieve data in a light-weight database on the user's device. This guide shows you how.

Some benefits of using SQLite for local storage

- ✓ SQLite is light-weight and self-contained. It's a code library without any other dependencies. There's nothing to configure.
- ✓ There's no database server. The client and the server run in the same process.
- ✓ SQLite is in the public domain so you can freely use and distribute it with your app.
- ✓ SQLite works across platforms and architectures.

You can read more about SQLite [here](#).

Choose an abstraction layer

We recommend that you use either the Entity Framework Core or the open-source [SQLite library](#) built by Microsoft.

Entity Framework Core

Entity Framework (EF) is an object-relational mapper that you can use to work with relational data by using domain-specific objects. If you've already used this framework to work with data in other .NET apps, you can migrate that code to a UWP app and it will work with appropriate changes to the connection string.

To try it out, see [Getting Started with EF Core](#).

SQLite library

The [Microsoft.Data.Sqlite](#) library implements the interfaces in the [System.Data.Common](#) namespace. Microsoft actively maintains these implementations, and they provide an intuitive wrapper around the low-level native SQLite API.

The rest of this guide helps you to use this library.

Set up your solution to use the Microsoft.Data.Sqlite library

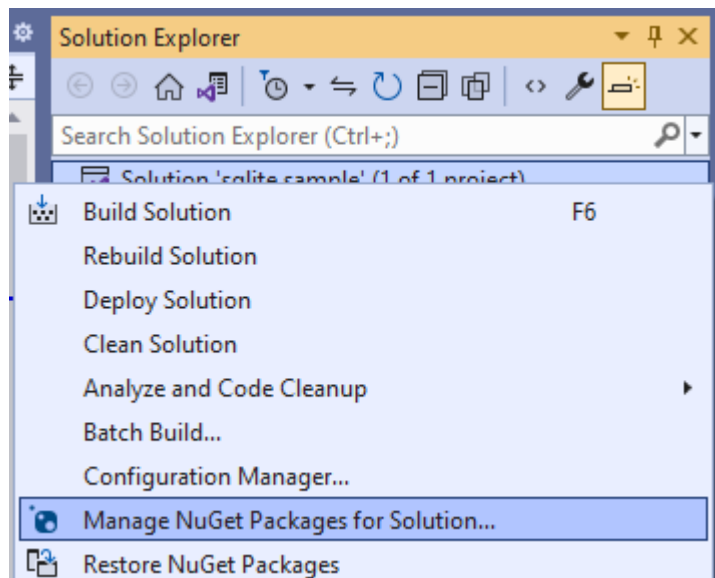
We'll start with a basic UWP project, and then install the appropriate NuGet packages.

All supported versions of Windows support SQLite, so your app does not have to package SQLite libraries. Instead, your app can use the version of SQLite that comes installed with Windows. This helps you in a few ways.

- ✓ Reduces the size of your application because you don't have to download the SQLite binary, and then package it as part of your application.
- ✓ Prevents you from having to push a new version of your app to users in the event that SQLite publishes critical fixes to bugs and security vulnerabilities in SQLite. The Windows version of SQLite is maintained by Microsoft in coordination with SQLite.org.
- ✓ App load time has the potential to be faster because most likely, the SDK version of SQLite will already be loaded into memory.

Let's start by adding a class to your UWP project named **DataAccess**. You can use a .NET Standard class library project to contain your data access code, but we won't use one in our example.

Right-click the solution, and then click **Manage NuGet Packages for Solution**.



At this point, you have a choice. You can use the version of SQLite that is included with Windows or if you have some reason to use a specific version of SQLite, you can include the SQLite library in your package. We are going to use the version of SQLite that's included with Windows.

Choose the **Browse** tab, search for the **Microsoft.Data.Sqlite.Core** package, and then install the latest stable version.

 **Microsoft.Data.Sqlite.Core** 



Versions - 0

☒	Project	Version	Installed
☒	sqlite.sample\sqlite.sample.csproj		

Installed:

Version:

Add and retrieve data in a SQLite database

We'll do these things:

- 1 Prepare the data access class.
- 2 Initialize the SQLite database.
- 3 Insert data into the SQLite database.
- 4 Retrieve data from the SQLite database.
- 5 Add a basic user interface.

Prepare the data access class

Open the `DataAccess` class in your project and make that class static.

Note

While our example will place data access code in a static class, it's just a design choice and is completely optional.

C#

```
public static class DataAccess
{

}
```

Add the following using statements to the top of this file.

```
C#

using Microsoft.Data.Sqlite;
using System.Collections.Generic;
```

Initialize the SQLite database

Add a method to the `DataAccess` class that initializes the SQLite database.

```
C#

public async static void InitializeDatabase()
{
    await
    ApplicationData.Current.LocalFolder.CreateFileAsync("sqliteSample.db",
    CreationCollisionOption.OpenIfExists);
    string dbpath = Path.Combine(ApplicationData.Current.LocalFolder.Path,
    "sqliteSample.db");
    using (SqliteConnection db =
    new SqliteConnection($"Filename={dbpath}"))
    {
        db.Open();

        String tableCommand = "CREATE TABLE IF NOT " +
        "EXISTS MyTable (Primary_Key INTEGER PRIMARY KEY, " +
        "Text_Entry NVARCHAR(2048) NULL)";

        SqliteCommand createTable = new SqliteCommand(tableCommand, db);

        createTable.ExecuteReader();
    }
}
```

This code creates the SQLite database and stores it in the application's local data store.

In this example, we name the database `sqliteSample.db` but you can use whatever name you want as long as you use that name in all `SqliteConnection` objects that you instantiate.

In the constructor of the **App.xaml.cs** file of your UWP project, call the `InitializeDatabase` method of the `DataAccess` class.

C#

```
public App()
{
    this.InitializeComponent();
    this.Suspending += OnSuspending;

    DataAccess.InitializeDatabase();
}
```

Insert data into the SQLite database

Add a method to the `DataAccess` class that inserts data into the SQLite database. This code uses parameters in the query to prevent SQL injection attacks.

C#

```
public static void AddData(string inputText)
{
    string dbpath = Path.Combine(ApplicationData.Current.LocalFolder.Path,
    "sqliteSample.db");
    using (SQLiteConnection db =
        new SQLiteConnection($"Filename={dbpath}"))
    {
        db.Open();

        SQLiteCommand insertCommand = new SQLiteCommand();
        insertCommand.Connection = db;

        // Use parameterized query to prevent SQL injection attacks
        insertCommand.CommandText = "INSERT INTO MyTable VALUES (NULL,
    @Entry)";
        insertCommand.Parameters.AddWithValue("@Entry", inputText);

        insertCommand.ExecuteReader();
    }
}
```

Retrieve data from the SQLite database

Add a method that gets rows of data from a SQLite database.

C#

```

public static List<String> GetData()
{
    List<String> entries = new List<string>();

    string dbpath = Path.Combine(ApplicationData.Current.LocalFolder.Path,
    "sqliteSample.db");
    using (SqliteConnection db =
        new SqliteConnection($"Filename={dbpath}"))
    {
        db.Open();

        SqliteCommand selectCommand = new SqliteCommand
            ("SELECT Text_Entry from MyTable", db);

        SqliteDataReader query = selectCommand.ExecuteReader();

        while (query.Read())
        {
            entries.Add(query.GetString(0));
        }
    }

    return entries;
}

```

The [Read](#) method advances through the rows of returned data. It returns `true` if there are rows left, otherwise it returns `false`.

The [GetString](#) method returns the value of the specified column as a string. It accepts an integer value that represents the zero-based column ordinal of the data that you want. You can use similar methods such as [GetDataTime](#) and [GetBoolean](#). Choose a method based on what type of data the column contains.

The ordinal parameter isn't as important in this example because we are selecting all of the entries in a single column. However, if multiple columns are part of your query, use the ordinal value to obtain the column you want to pull data from.

Add a basic user interface

In the **MainPage.xaml** file of the UWP project, add the following XAML.

XML

```

<Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
    <StackPanel>
        <TextBox Name="Input_Box"></TextBox>
        <Button Click="AddData">Add</Button>
        <ListView Name="Output">

```

```

        <ListView.ItemTemplate>
            <DataTemplate>
                <TextBlock Text="{Binding}" />
            </DataTemplate>
        </ListView.ItemTemplate>
    </ListView>
</StackPanel>
</Grid>

```

This basic user interface gives the user a `TextBox` that they can use to type a string that we'll add to the SQLite database. We'll connect the `Button` in this UI to an event handler that will retrieve data from the SQLite database and then show that data in the `ListView`.

In the `MainPage.xaml.cs` file, add the following handler. This is the method that we associated with the `Click` event of the `Button` in the UI.

```

C#

private void AddData(object sender, RoutedEventArgs e)
{
    DataAccess.AddData(Input_Box.Text);

    Output.ItemsSource = DataAccess.GetData();
}

```

We also want to be sure that any existing data is loaded when the application starts. Add a line of code to the `MainPage` constructor to call `GetData()`.

```

C#

public MainPage()
{
    this.InitializeComponent();

    Output.ItemsSource = DataAccess.GetData();
}

```

That's it. Explore the [Microsoft.Data.Sqlite](#) to see what other things you can do with your SQLite database. Check out the links below to learn about other ways to use data in your UWP app.

Next steps

Connect your app directly to a SQL Server database

See [Use a SQL Server database in a UWP app](#).

Share code between different apps across different platforms

See [Share code between desktop and UWP](#).

Add master detail pages with Azure SQL back ends

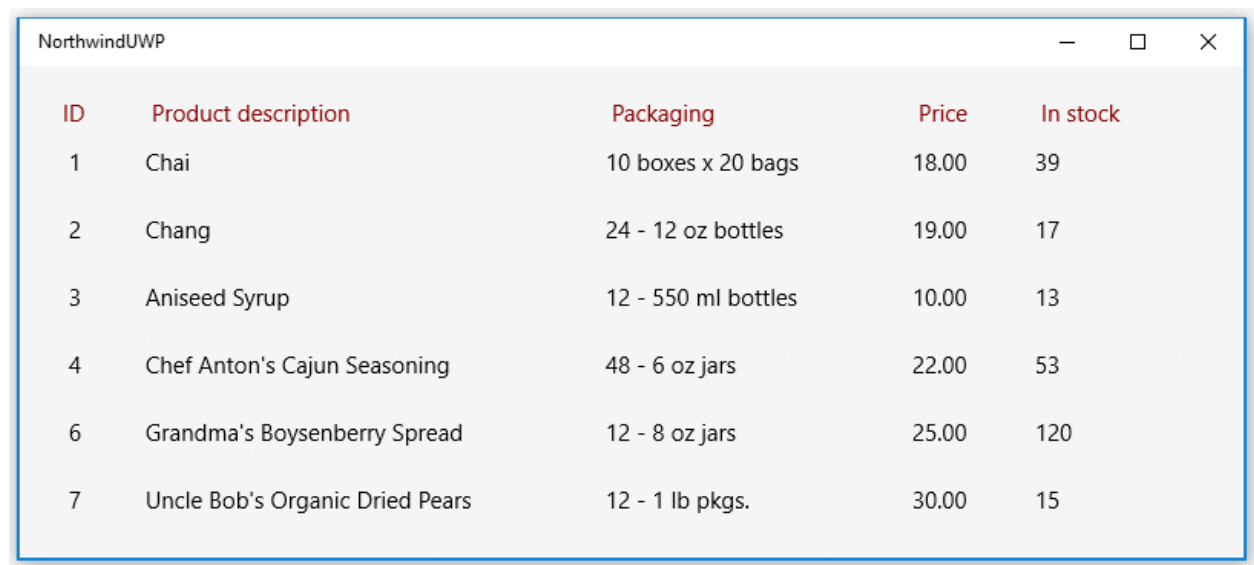
See [Customer Orders Database sample](#) [↗](#).

Use a SQL Server database in a UWP app

Article • 10/15/2024

Your app can connect directly to a SQL Server database and then store and retrieve data by using classes in the [System.Data.SqlClient](#) namespace.

In this guide, we'll show you one way to do that. If you install the [Northwind](#) sample database onto your SQL Server instance, and then use these snippets, you'll end up with a basic UI that shows products from the Northwind sample database.



ID	Product description	Packaging	Price	In stock
1	Chai	10 boxes x 20 bags	18.00	39
2	Chang	24 - 12 oz bottles	19.00	17
3	Aniseed Syrup	12 - 550 ml bottles	10.00	13
4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	22.00	53
6	Grandma's Boysenberry Spread	12 - 8 oz jars	25.00	120
7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	30.00	15

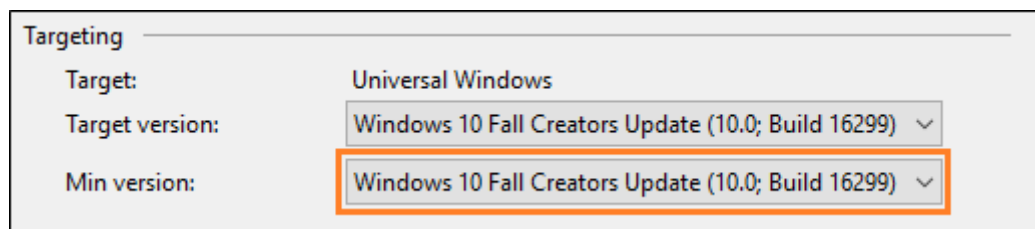
💡 Tip

You can also get scripts to create the **Northwind** and **pubs** sample databases from the [SQL Server Samples GitHub repository](#).

The snippets that appear in this guide are based on this more [complete sample](#).

First, set up your solution

To connect your app directly to a SQL Server database, make sure that the minimum version of your project targets the Windows 10 Fall Creators update (Build 16299) or newer. You can find that information in the properties page of your UWP project.



Targeting

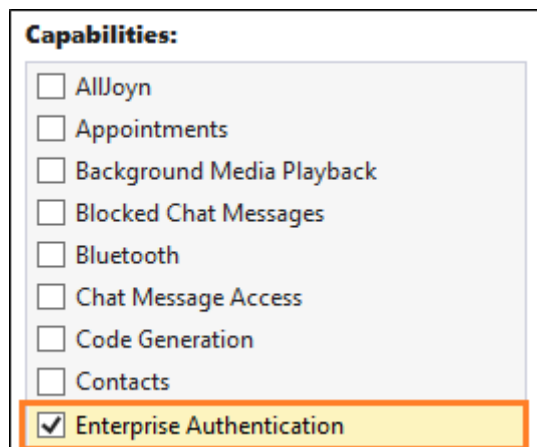
Target: Universal Windows

Target version: Windows 10 Fall Creators Update (10.0; Build 16299) ▾

Min version: Windows 10 Fall Creators Update (10.0; Build 16299) ▾

Open the **Package.appxmanifest** file of your UWP project in the manifest designer.

In the **Capabilities** tab, select the **Enterprise Authentication** checkbox if you are using Windows Authentication for authenticating your SQL Server.



Capabilities:

- ☐ AllJoyn
- ☐ Appointments
- ☐ Background Media Playback
- ☐ Blocked Chat Messages
- ☐ Bluetooth
- ☐ Chat Message Access
- ☐ Code Generation
- ☐ Contacts
- ☒ Enterprise Authentication

Important

You will also need to select **Internet (Client & Server)**, **Internet (Client)**, and **Private Networks (Client & Server)**, regardless of whether or not you're using Windows Authentication.

Add and retrieve data in a SQL Server database

In this section, we'll do these things:

- 1 Add a connection string.
- 2 Create a class to hold product data.
- 3 Retrieve products from the SQL Server database.
- 4 Add a basic user interface.
- 5 Populate the UI with Products.

Note

This section illustrates one way to organize your data access code. It's meant only to provide an example of how you can use [System.Data.SqlClient](#) to store and retrieve data from a SQL Server database. You can organize your code in any way that makes the most sense to your application's design.

Add a connection string

In the **App.xaml.cs** file, add a property to the `App` class, that gives other classes in your solution access to the connection string.

Our connection string points to the Northwind database in a SQL Server Express instance. The connection string in this snippet assumes you kept the default instance name `SQLEXPRESS` when installing SQL Server Express. You can make changes to the connection string to match your SQL Server instance, database, and authentication method.

C#

```
sealed partial class App : Application
{
    // Create a connection string using Windows Authentication.
    private string connectionString =
        @"Data Source=.\SQLEXPRESS;Initial Catalog=NORTHWIND;Integrated
        Security=SSPI";

    public string ConnectionString { get => connectionString; set =>
        connectionString = value; }

    ...
}
```

Important

In production applications, connection information should be stored securely in app configuration (see [Adding Azure App Configuration by using Visual Studio Connected Services](#)). Connection strings and other secrets should never be hard-coded.

Create a class to hold product data

We'll create a class that implements the [INotifyPropertyChanged](#) event so that we can bind attributes in our XAML UI to the properties in this class.

C#

```
public class Product : INotifyPropertyChanged
{
    public int ProductID { get; set; }
    public string ProductCode { get { return ProductID.ToString(); } }
    public string ProductName { get; set; }
    public string QuantityPerUnit { get; set; }
    public decimal UnitPrice { get; set; }
    public string UnitPriceString { get { return
UnitPrice.ToString("#####.00"); } }
    public int UnitsInStock { get; set; }
    public string UnitsInStockString { get { return
UnitsInStock.ToString("#####0"); } }
    public int CategoryId { get; set; }

    public event PropertyChangedEventHandler PropertyChanged;
    private void NotifyPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }
}
```

Retrieve products from the SQL Server database

In the **MainPage.xaml.cs** file of the UWP project, create a method that gets products from the Northwind sample database, and then returns them as an

[ObservableCollection](#) collection of `Product` instances.

C#

```
public ObservableCollection<Product> GetProducts(string connectionString)
{
    const string GetProductsQuery = "select ProductID, ProductName,
QuantityPerUnit," +
        " UnitPrice, UnitsInStock, Products.CategoryID " +
        " from Products inner join Categories on Products.CategoryID =
Categories.CategoryID " +
        " where Discontinued = 0";

    var products = new ObservableCollection<Product>();
    try
    {
        using (var conn = new SqlConnection(connectionString))
        {
            conn.Open();
            if (conn.State == System.Data.ConnectionState.Open)
            {
                using (SqlCommand cmd = conn.CreateCommand())
```

```

    {
        cmd.CommandText = GetProductsQuery;
        using (SqlDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                var product = new Product();
                product.ProductID = reader.GetInt32(0);
                product.ProductName = reader.GetString(1);
                product.QuantityPerUnit = reader.GetString(2);
                product.UnitPrice = reader.GetDecimal(3);
                product.UnitsInStock = reader.GetInt16(4);
                product.CategoryId = reader.GetInt32(5);
                products.Add(product);
            }
        }
    }
}
return products;
}
catch (Exception eSql)
{
    Debug.WriteLine($"Exception: {eSql.Message}");
}
return null;
}

```

Add a basic user interface

Add the following XAML to the **MainPage.xaml** file of the UWP project.

This XAML creates a [ListView](#) to show each product that you returned in the previous snippet, and binds the attributes of each row in the [ListView](#) to the properties that we defined in the `Product` class.

XML

```

<Grid Background="{ThemeResource SystemControlAcrylicWindowBrush}">
    <RelativePanel>
        <ListView Name="InventoryList"
            SelectionMode="Single"
            ScrollViewer.VerticalScrollBarVisibility="Auto"
            ScrollViewer.IsVerticalRailEnabled="True"
            ScrollViewer.VerticalScrollMode="Enabled"
            ScrollViewer.HorizontalScrollMode="Enabled"
            ScrollViewer.HorizontalScrollBarVisibility="Auto"
            ScrollViewer.IsHorizontalRailEnabled="True"
            Margin="20">
            <ListView.HeaderTemplate>
                <DataTemplate>

```

```

        <StackPanel Orientation="Horizontal" >
            <TextBlock Text="ID" Margin="8,0" Width="50"
Foreground="DarkRed" />
            <TextBlock Text="Product description" Width="300"
Foreground="DarkRed" />
            <TextBlock Text="Packaging" Width="200"
Foreground="DarkRed" />
            <TextBlock Text="Price" Width="80"
Foreground="DarkRed" />
            <TextBlock Text="In stock" Width="80"
Foreground="DarkRed" />
        </StackPanel>
    </DataTemplate>
</ListView.HeaderTemplate>
<ListView.ItemTemplate>
    <DataTemplate x:DataType="local:Product">
        <StackPanel Orientation="Horizontal" >
            <TextBlock Name="ItemId"
                Text="{x:Bind ProductCode}"
                Width="50" />
            <TextBlock Name="ItemName"
                Text="{x:Bind ProductName}"
                Width="300" />
            <TextBlock Text="{x:Bind QuantityPerUnit}"
                Width="200" />
            <TextBlock Text="{x:Bind UnitPriceString}"
                Width="80" />
            <TextBlock Text="{x:Bind UnitsInStockString}"
                Width="80" />
        </StackPanel>
    </DataTemplate>
</ListView.ItemTemplate>
</ListView>
</RelativePanel>
</Grid>

```

Show products in the ListView

Open the **MainPage.xaml.cs** file, and add code to the constructor of the **MainPage** class that sets the **ItemSource** property of the **ListView** to the **ObservableCollection** of **Product** instances.

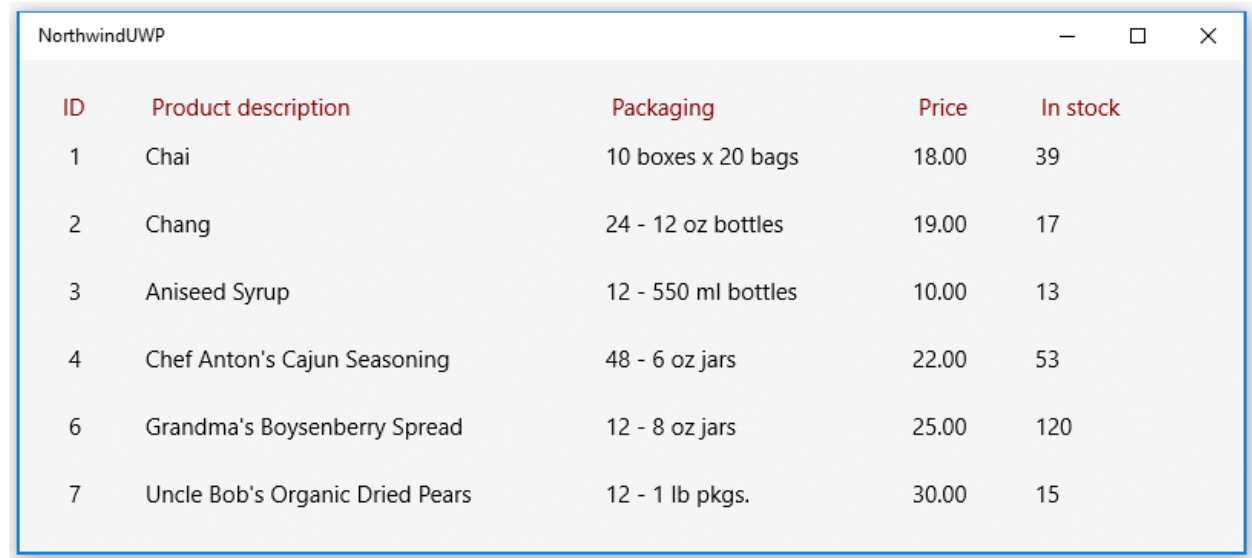
C#

```

public MainPage()
{
    this.InitializeComponent();
    InventoryList.ItemsSource = GetProducts((App.Current as
App).ConnectionString);
}

```

Start the project and see products from the Northwind sample database appear in the UI.

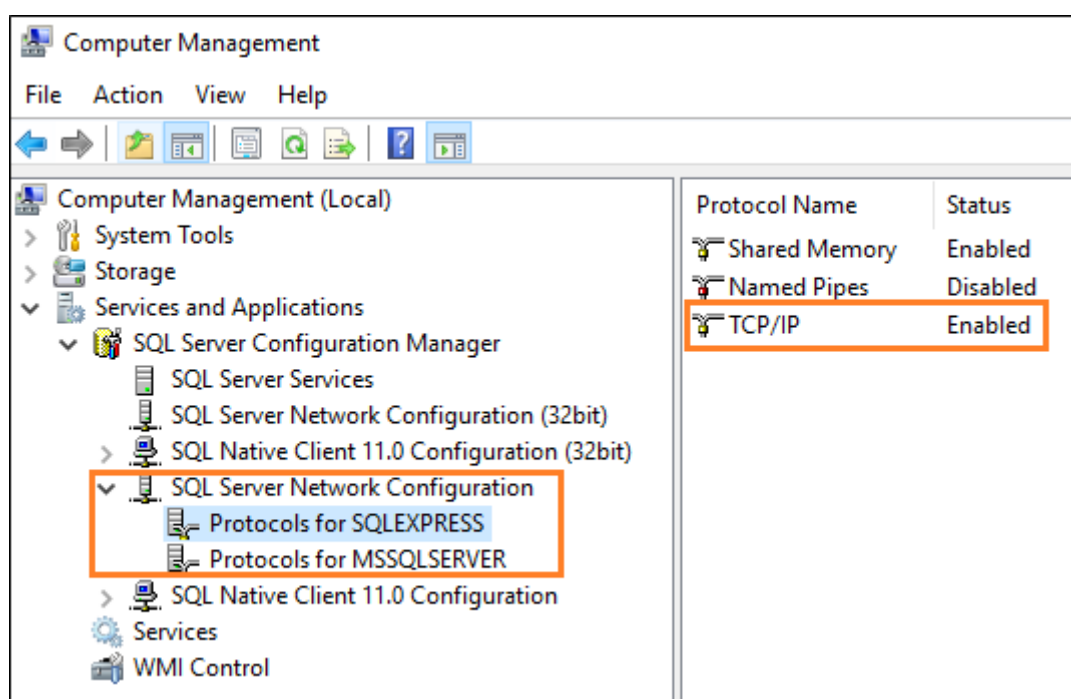


ID	Product description	Packaging	Price	In stock
1	Chai	10 boxes x 20 bags	18.00	39
2	Chang	24 - 12 oz bottles	19.00	17
3	Aniseed Syrup	12 - 550 ml bottles	10.00	13
4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	22.00	53
6	Grandma's Boysenberry Spread	12 - 8 oz jars	25.00	120
7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	30.00	15

Explore the [System.Data.SqlClient](#) namespace to see what other things you can do with data in your SQL Server database.

Trouble connecting to your database?

In most cases, some aspect of the SQL Server configuration needs to be changed. If you're able to connect to your database from another type of desktop application such as a Windows Forms or WPF application, ensure that you've enabled TCP/IP for SQL Server. You can do that in the **Computer Management** console. (See [Windows Tools/Administrative Tools](#) for more information.)



Then, make sure that your SQL Server Browser service is running.